



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

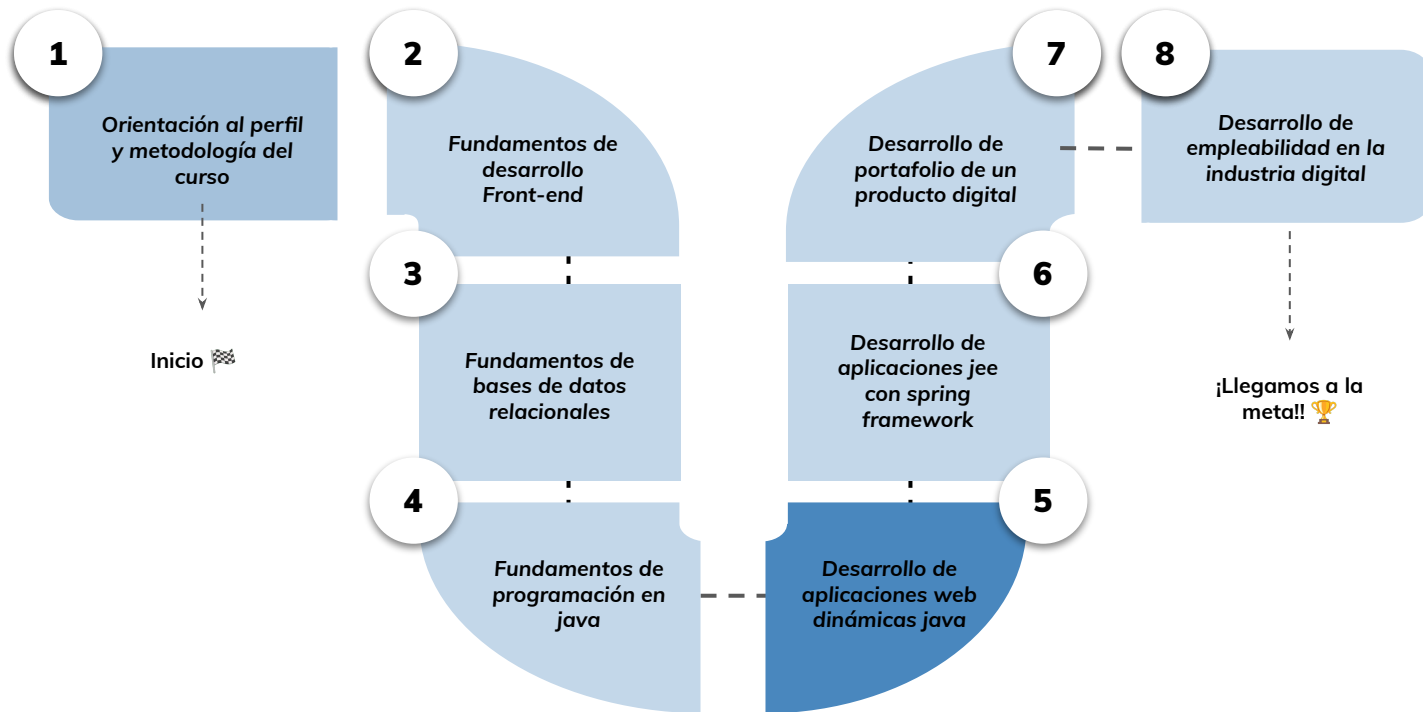


› JEE y su entorno- Parte 1

Aprendizaje Esperado 1: Caracterizar los elementos fundamentales que componen una aplicación web dinámica y su entorno de desarrollo utilizando tecnologías j2ee.

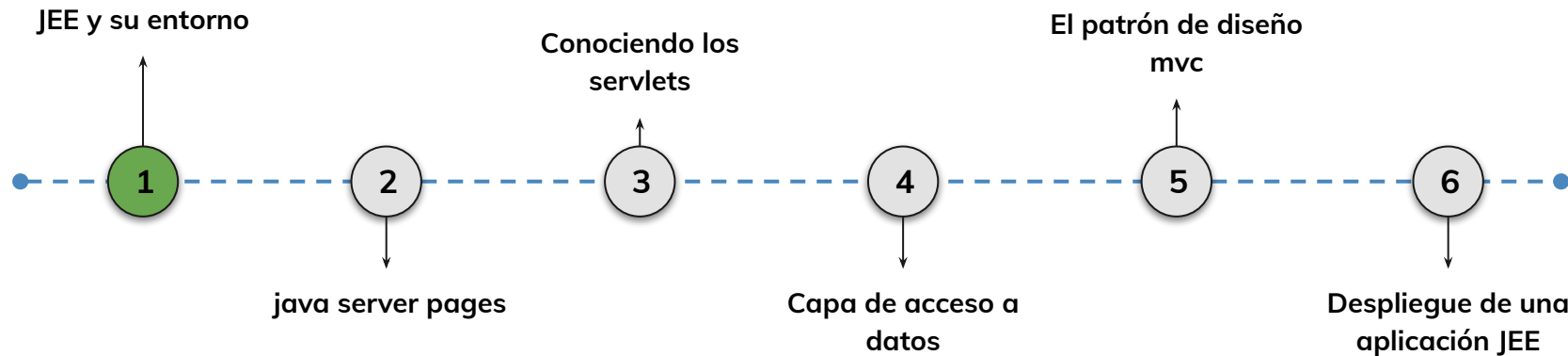
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles *lecciones* estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

1.

Aplicaciones web dinámicas con JEE

Exploraremos los fundamentos del desarrollo de aplicaciones web dinámicas utilizando la arquitectura Java EE, incluyendo sus capas, componentes, y el uso de servlets, JSP, JSF y servidores de aplicaciones.

Arquitectura Java EE

Entorno de ejecución

Capas de una aplicación JEE (cliente, web, negocio, persistencia)

Componentes clave: Servlets, JSP, JSF, EJB

Servidores de aplicaciones: Apache Tomcat, JBoss, WebLogic

Ciclo de vida de los Servlets y ejemplo de funcionamiento

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender qué es Java EE y su propósito en aplicaciones empresariales.
- Reconocer la arquitectura en capas de una aplicación web dinámica.
- Identificar y describir los componentes principales de JEE (Servlets, JSP, JSF, EJB).
- Explicar cómo interactúan las capas cliente, web, negocio y persistencia.
- Comprender el rol de los servidores de aplicaciones y su impacto en disponibilidad, escalabilidad y mantenimiento.

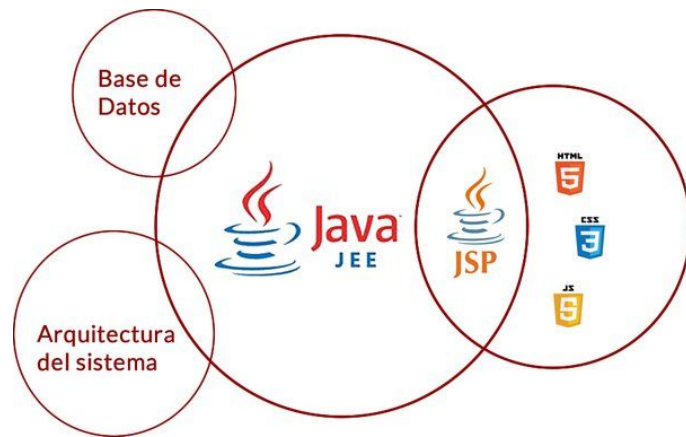
➤ El Entorno JEE

< > El Entorno JEE

¿Qué es?

Java Enterprise Edition o JEE es una plataforma de programación para desarrollar y ejecutar aplicaciones en entorno Java, **basada en arquitectura multicapa utilizando componentes modulares**.

JEE define las APIs y requerimientos necesarios para poder ejecutar aplicaciones Java servidoras, con todo lo que ello supone: cliente servidor, multiusuario, transaccionalidad, escalabilidad, etc.



< > El Entorno JEE

Arquitectura JEE:

La arquitectura Java EE está basada en tres conceptos clave:

- **Servicios** : Permiten que el programador se concentre en su lógica de negocio y usar los servicios para su aplicación. Estos servicios son proporcionados por un contenedor.
- **Contenedores** : Son entornos en tiempo de ejecución; es decir, un programa que se está ejecutando y tu aplicación se monta sobre este como si fuera un plugin o el casete para una consola de juegos.
- **Componentes** : Son objetos Java que contienen la lógica de negocio de la aplicación y usan los servicios proporcionados por el contenedor. Hay varios tipos de componentes y según ese tipo son desplegados en un contenedor u otro.



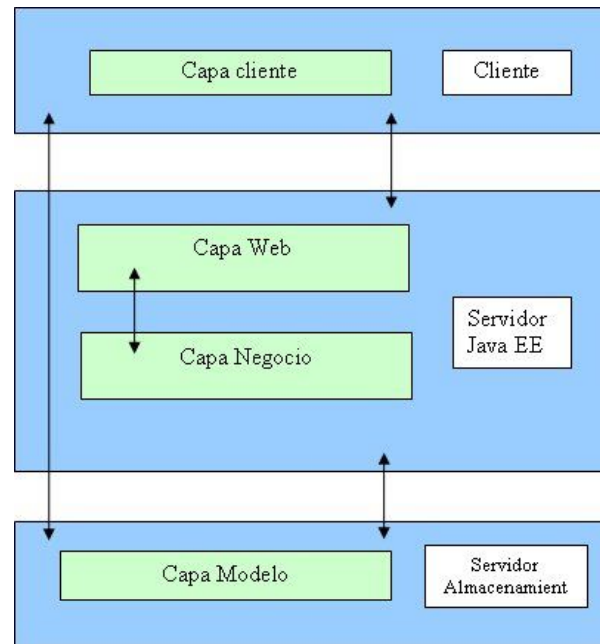
< > El Entorno JEE

Arquitectura en Capas:

Estos tres conceptos permiten a Java EE definir una arquitectura de capas.

Las aplicaciones Java EE multicapa estándar consisten en 3 o 4 capas:

- Capa cliente
- Capa web
- Capa de negocio/servicio
- Capa de almacenamiento/persistencia



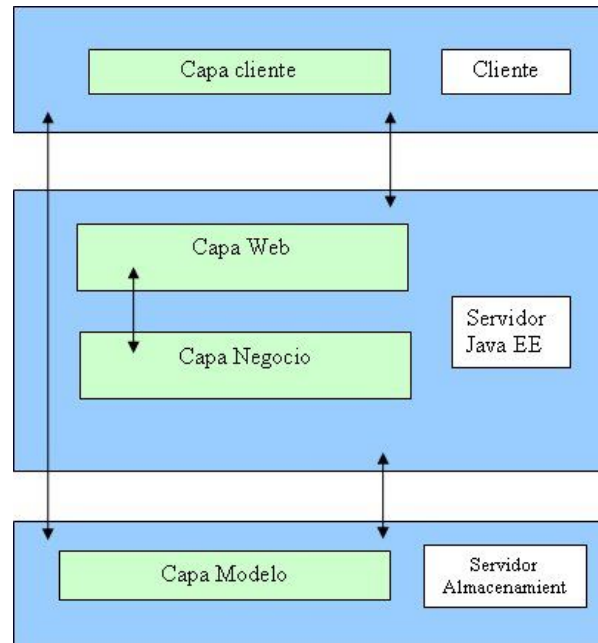
< > El Entorno JEE

Capa Cliente:

Un cliente JEE normalmente es un cliente web o una aplicación cliente.

Este software está instalado y se ejecuta siempre en el ordenador del usuario y no en nuestro servidor. En principio no debemos responsabilizarnos de este software, pero sí tendremos que tenerlo en cuenta, ya que cada uno posee su propia especificación.

Por ejemplo, no todos los navegadores web decodifican igual el código HTML, por lo que una página que funcione correctamente para Internet Explorer puede no hacerlo para Google Chrome.



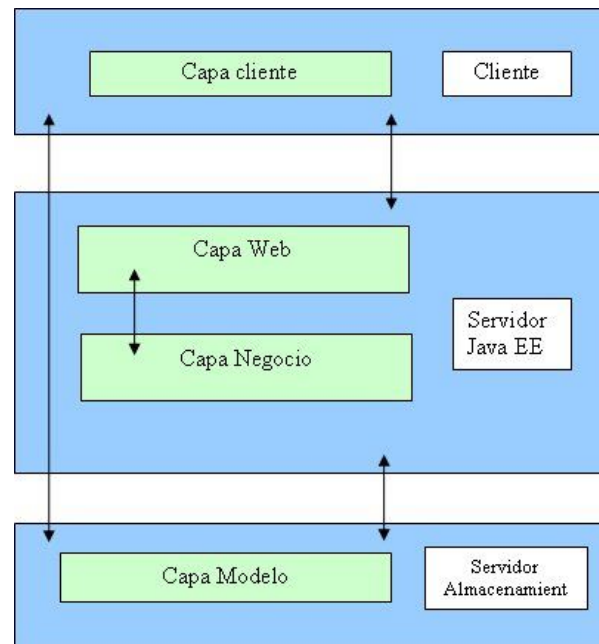
< > El Entorno JEE

Capa Web:

Esta capa está compuesta por servlets, páginas web creadas con tecnologías como JavaServer Faces o JSP y otros componentes que interactúan entre la capa cliente y la capa de negocio.

Un Servlet es una clase Java que procesa peticiones dinámicamente y construye las respuestas,

Las páginas JSP o JSF son páginas que permiten que parte de su código se ejecute como si fuera un servlet y otra parte pueda ser texto estático que nos facilite la visualización de los datos.

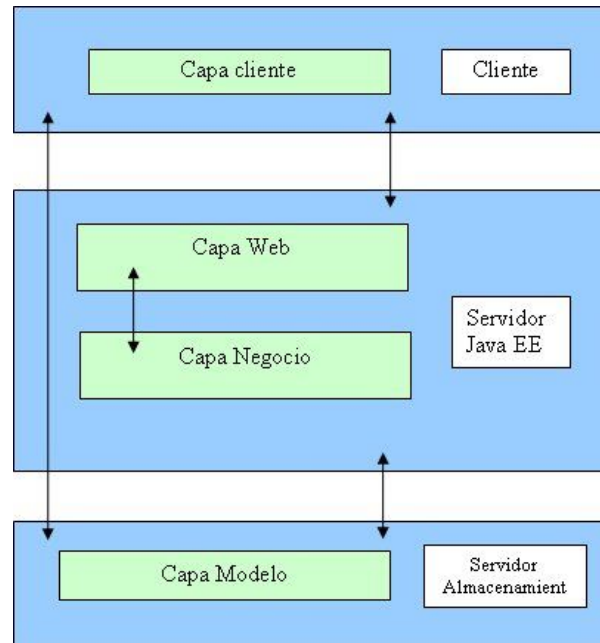


< > El Entorno JEE

Capa de negocio:

Contiene los componentes que forman la lógica de negocio de la aplicación.

Se comunica con la capa web de la que recibe la información del cliente y con la capa de almacenamiento de la que recibe los datos almacenados en los sistemas de información de la empresa.



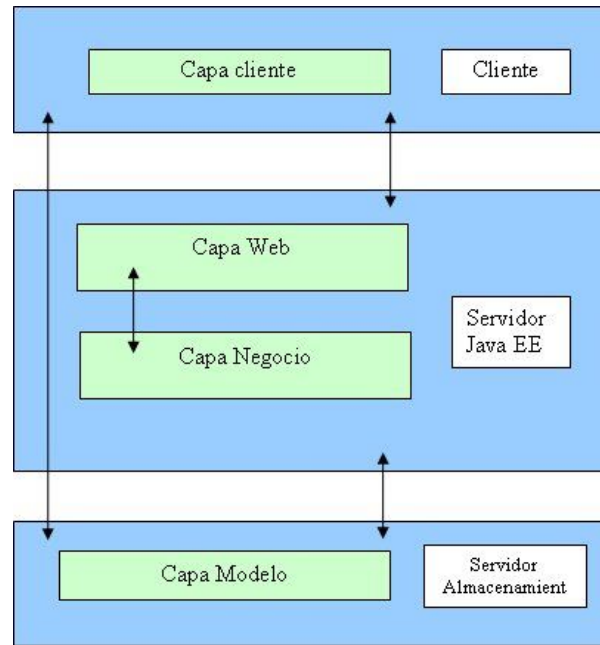
< > El Entorno JEE

Capa de almacenamiento:

Está formada por los sistemas de almacenamiento de información de la empresa. Pueden ser Bases de Datos, sistemas de planificación de recursos (ERP), BBDD, sistemas de almacenamiento de datos, ficheros XML, etc.

En esta organización está basada la arquitectura MVC que distribuye las aplicaciones en 3 capas (Modelo, Vista y Controlador).

La nueva especificación Java EE amplía el diseño a **n** capas, pudiendo extender cada capa en diferentes subcapas pero manteniendo la idea original.



< > El Entorno JEE

Algunos de los componentes clave en el entorno JEE incluyen:

- **Java Servlets** : Son componentes de servidor que manejan las solicitudes HTTP y generan respuestas dinámicas.
- **JavaServer Pages (JSP)** : Son tecnologías de presentación que permiten mezclar contenido HTML estático con código Java para generar páginas web dinámicas.
- **JavaServer Faces (JSF)** : Es un framework de componentes de interfaz de usuario basado en Java para la construcción de aplicaciones web.
- **Enterprise JavaBeans (EJB)** : Son componentes de negocio distribuidos que encapsulan la lógica empresarial de una aplicación.



< > El Entorno JEE

¿Qué es una JavaServer Page (JSP)?

Es una tecnología que nos permite **mezclar HTML estático con HTML generado dinámicamente** .

JSP es un acrónimo de Java Server Pages, que en castellano vendría a decir algo como Páginas de Servidor Java. Es, pues, una tecnología orientada a crear páginas web con programación en Java.

Con JSP podemos crear aplicaciones web que se ejecuten en variados servidores web, de múltiples plataformas, ya que Java es en esencia un lenguaje multiplataforma. Las páginas JSP están compuestas de código HTML/XML mezclado con etiquetas especiales para programar scripts de servidor en sintaxis Java. Por tanto, las JSP podremos escribirlas con nuestro editor HTML/XML habitual.



➤ Servlet

< > Servlet

¿Qué es?

Es una tecnología que nos permite crear aplicaciones web interactivas (dinámicas), es decir, le permite al usuario interactuar con la aplicación (hacer consultas, insertar datos, modificarlos, etc). Un Servlet es un objeto Java que pertenece a una clase que extiende de `javax.servlet.http.HttpServlet`.

Son pequeños programas escritos en Java que admiten peticiones a través del protocolo HTTP. Los servlets reciben peticiones desde un navegador web, las procesan y devuelven una respuesta al navegador, normalmente en HTML.

Estos programas son los intermediarios entre el cliente (casi siempre un navegador web) y los datos (BBDD).

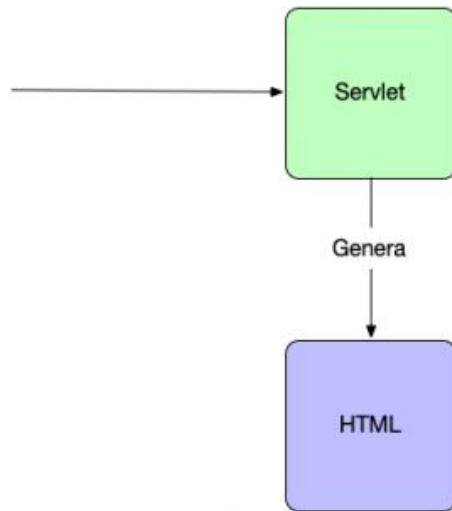


< > Servlet

Ciclo de vida:

Desde el momento que inicializamos el servlet hasta que el servlet es destruido, este pasará por una serie de estados dependiendo de cada una de las situaciones ante las que se encuentre. Muy por encima podríamos decir que la secuencia de acciones que se producen en un servlet son las siguientes:

- La primera vez que realicemos una petición sobre el servlet se ejecutará un método de inicio.
- Luego estaremos a la espera de peticiones. Dependiendo de como lleguen los datos al servlet se ejecutará un método específico.
- Por último, el servlet tendrá un estado de finalización en el cual eliminará las variables creadas en su inicialización, conexiones a bases de datos, etc. Finalmente se destruirá.

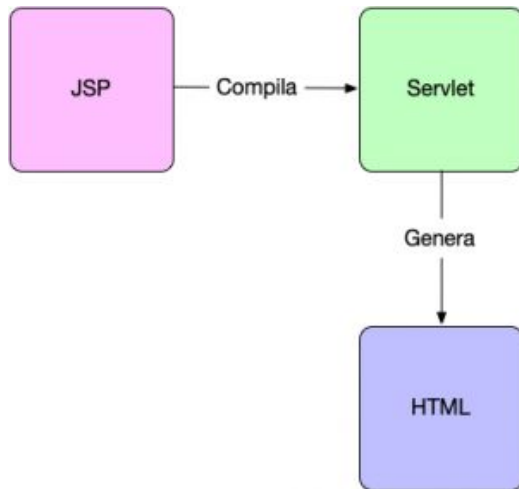


< > Servlet

¿Cómo funciona?

1. El navegador (cliente) pide una página al servidor HTTP que es un contenedor de Servlets.
2. El servlet procesa los argumentos de la petición, es decir, el contenedor de Servlets delega la petición a un Servlet en particular elegido de entre los Servlets que contiene.
3. El Servlet, que es una objeto java, se encarga de generar el texto de la página web que se entrega al contenedor.
4. El contenedor devuelve la página web al navegador (cliente) que la solicitó, normalmente en HTML.

Por lo tanto, nos encontramos en una arquitectura Cliente-Servidor. En nuestro caso vamos a utilizar Apache Tomcat como contenedor de servlets.



➤ Servidor de aplicaciones

< > Servidor de aplicaciones

¿Qué es?

Los servidores de aplicaciones **son programas de servidor en una red distribuida que proporciona el entorno de ejecución para un programa de aplicación** . Más específicamente, el servidor de aplicaciones es el componente de tiempo de ejecución principal en todas las configuraciones y donde una aplicación se ejecuta realmente.

El servidor de aplicaciones colabora con el servidor web para ofrecer una respuesta dinámica y personalizada a una solicitud de cliente.

Algunos ejemplos populares de servidores de aplicaciones JEE son *Apache Tomcat*, *JBoss/WildFly*, *IBM WebSphere Application Server* y *Oracle WebLogic Server*.



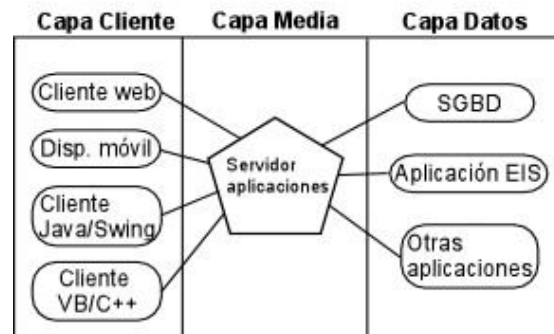
**APACHE
TOMCAT**



< > Servidor de aplicaciones

El concepto de servidor de aplicaciones está relacionado con el concepto de sistema distribuido. Un sistema distribuido, en oposición a un sistema monolítico, permite mejorar tres aspectos fundamentales en una aplicación: la alta disponibilidad, la escalabilidad y el mantenimiento.

- La alta disponibilidad hace referencia a que un sistema debe estar funcionando las 24 horas del día los 365 días al año.
- La escalabilidad es la capacidad de hacer crecer un sistema cuando se incrementa la carga de trabajo (el número de peticiones).
- El mantenimiento tiene que ver con la versatilidad a la hora de actualizar, depurar fallos y mantener un sistema. La solución al mantenimiento es la construcción de la lógica de negocio en unidades reusables y modulares.





LIVE CODING

Ejemplo en vivo

Subtítulo:

Aplicaciones web con JEE

1. *Cómo crear una aplicación web*
2. *Cómo crear Servlets*
3. *Crear método DO POST y método DO GET para recibir peticiones*

Tiempo: 1 hs

Live Coding

¿En qué consistirá la Demo?

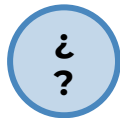
Crear un *Dynamic Web Project* en Eclipse e implementar un Servlet (*/hola*) con *doGet* y *doPost*.

1. Crear proyecto en Eclipse (Generate web.xml).
2. Configurar Tomcat 9 como Runtime.
3. Nuevo Servlet *HelloServlet* con `@WebServlet("/hola")`.
4. Codificar *doGet*: leer `?nombre=` y responder HTML.
5. Crear *index.jsp* con formulario GET a */hola*.
6. Agregar *doPost*: leer *msg* y mostrar respuesta.
7. Run on Server → abrir `http://localhost:8080/<app>/index.jsp`.

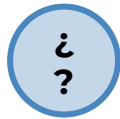
#Momentode Preguntas...



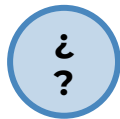
¿Qué ventajas ofrece Java EE frente a una aplicación monolítica tradicional?



¿Qué diferencias encuentras entre un Servlet y una JSP?



¿Por qué crees que es importante separar las capas de una aplicación?



¿Qué desafíos encuentras al implementar Servlets sin un contenedor real?



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Mapa conceptual del entorno JEE

Mapa conceptual del entorno JEE

Contexto: 🙌

Estás iniciando un proyecto en equipo donde deben decidir si usar Java EE como base para desarrollar una aplicación web dinámica. El equipo necesita una descripción clara y visual de cómo se compone esta tecnología, qué elementos la integran, cómo se organizan, y cuál es su propósito dentro del entorno de desarrollo. Para ello, te encomiendan armar una caracterización completa.

Consigna: 📝

A partir del contenido visto en clase, elabora un mapa conceptual o esquema visual que caracterice los elementos fundamentales del entorno JEE. Deberás incluir las capas de la arquitectura (cliente, web, negocio, persistencia), describir sus funciones, y representar los componentes principales (Servlet, JSP, JSF, EJB, contenedores, servicios, etc.). El trabajo debe explicar qué son, cómo se relacionan y qué rol cumplen.

Tiempo 🕒: 40 minutos

Paso a paso: ⚙️

1. Lee nuevamente el material de clase (Parte 1) sobre arquitectura y componentes de JEE.
2. Usa una herramienta gratuita y sin cuenta como:
 - a. 📌 <https://app.diagrams.net> o
 - b. 📌 <https://excalidraw.com>
3. Crea un esquema visual en el que:
 - a. Identifiques las capas de JEE
 - b. Coloques los componentes en la capa que les corresponda
 - c. Incluyas una breve descripción de cada elemento
 - d. Conectes los elementos usando flechas o líneas con sentido lógico
4. Guarda el archivo o expórtalo como imagen/PDF para subirlo a la plataforma.

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ Entender qué es Java EE y cómo organiza una aplicación web
- ✓ Conocer las capas: cliente, web, negocio, persistencia
- ✓ Comprender el funcionamiento de Servlets y JSP
- ✓ Reconocer el rol de los servidores de aplicaciones
- ✓ Aplicar un ejemplo práctico sencillo sin entorno de servidor real
- ✓ Conectar teoría y práctica con herramientas accesibles
- ✓ Sentar base para siguientes temas más complejos



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son claves para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

